Buổi 2

Kỹ thuật tìm kiếm (Searching)

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Khái quát về tìm kiếm

1.1 Bài toán tìm kiếm

2. Các giải thuật tìm kiếm nội

2.1 Tìm tuyến tính
2.2 Tìm tuyến tính – phân tích và minh họa
2.3 Cải tiến: tìm tuyến tính với phần tử cầm canh
2.4 Tìm nhị phân
2.5 Tìm nhị phân không đệ quy và phân tích
2.6 Minh họa tìm nhị phân
2.7 Các biến thể và lỗi thường gặp của tìm nhị phân
2.8 So sánh tìm tuyến tính và tìm nhị phân

3. Các giải thuật tìm kiếm ngoại

3.1 Đặt vấn đề tìm kiếm trên tập tin
3.2 Tìm tuyến tính trên tập tin
3.3 Tìm kiếm theo chỉ mục
3.4 Phân tích tìm kiếm theo chỉ mục

4. Tổng kết và thực hành

4.1 Tóm tắt và bài thực hành buổi 2
4.2 Câu hỏi và bài tập

Bài toán

Cho dãy M gồm N phần tử, mỗi phần tử có một **khóa** (key). Cho giá trị X. Tìm phần tử có khóa bằng X: trả về **vị trí** (chỉ số) nếu tìm thấy, ngược lại trả về **−1**.

- **Khóa**: thành phần dùng để phân biệt/so sánh các phần tử (mã SV, số điện thoại…). Để đơn giản, giáo trình xét mảng các số nguyên với khóa chính là giá trị phần tử: `T M[]`, `int N`, `T X`.
- **Phân loại:**
  - **Tìm kiếm nội**: dữ liệu nằm trong **bộ nhớ trong** (mảng, danh sách).
  - **Tìm kiếm ngoại**: dữ liệu nằm trên **bộ nhớ ngoài** (tập tin) – chi phí chủ yếu là số lần **đọc đĩa**.
- **Đánh giá** thuật toán tìm kiếm: đếm số **phép so sánh** (S) và số **phép gán** (G) trong 3 trường hợp: tốt nhất, xấu nhất, trung bình.
- **Ứng dụng:** tra danh bạ, tìm SV theo mã, kiểm tra tài khoản đăng nhập, tìm từ trong từ điển…

Ví dụ 2.1 – Dãy dùng xuyên suốt buổi học

M = {1, 3, 4, 5, 8, 15, 17, 22, 25, 30}, N = 10 (chỉ số 0..9).

- Tìm X = 5 ⇒ kết quả **3**.
- Tìm X = 7 ⇒ kết quả **−1**.

2.1 Tìm tuyến tính (Linear Search) – tư tưởng và thuật toán

- **Tư tưởng:** lần lượt so sánh X với các phần tử M[1], M[2], … cho đến khi gặp phần tử bằng X hoặc **hết dãy**. Không đòi hỏi dãy có thứ tự.

- **Thuật toán:**

```
B1: k = 1                  // bắt đầu từ phần tử đầu
B2: IF M[k] ≠ X AND k ≤ N
      B2.1: k++
      B2.2: Lặp lại B2
B3: IF k ≤ N
      Tìm thấy tại vị trí k
B4: ELSE
      Không tìm thấy phần tử có giá trị X
B5: Kết thúc
```

- **Cài đặt:**

```
int LinearSearch(T M[], int N, T X)
{  int k = 0;
   while (M[k] != X && k < N)
      k++;
   if (k < N)
      return (k);
   return (-1);
}
```

- **Lưu ý thứ tự điều kiện:** nên viết `k < N && M[k] != X` để không truy cập M[N] **ngoài mảng** khi không tìm thấy (C/C++ đánh giá điều kiện tắt từ trái sang phải).

2.2 Tìm tuyến tính – phân tích và minh họa

- **Phân tích (thuật toán nguyên thủy):**
  - Tốt nhất – M[0] = X: Gmin = 1; Smin = 2 + 1 = **3**.
  - Xấu nhất – không tìm thấy: Gmax = N + 1; Smax = 2N + 1 + 1 = **2N + 2**.
  - Trung bình: Gavg = (N + 2)/2; Savg = (3 + 2N + 2)/2 = **N + 2,5**.
- ⇒ Độ phức tạp **O(N)**.

Ví dụ 2.2 – Tìm X = 15 trong M

k	M[k]	M[k] ≠ X?	Hành động
0	1	Đúng	k = 1
1	3	Đúng	k = 2
2	4	Đúng	k = 3
3	5	Đúng	k = 4
4	8	Đúng	k = 5
5	15	**Sai**	Dừng ⇒ trả về **5**

- Số phép so sánh trong vòng lặp: 6 × 2 = 12, cộng 1 phép so sánh `k < N` cuối ⇒ **13**.

Ưu, nhược điểm

Ưu: đơn giản, áp dụng cho **mọi** dãy (không cần sắp xếp), mọi cấu trúc duyệt tuần tự được (mảng, danh sách liên kết, tập tin).
Nhược: chậm khi N lớn.

2.3 Cải tiến: tìm tuyến tính với phần tử cầm canh

- **Ý tưởng:** đặt thêm X vào **cuối** dãy (M[N] = X) – gọi là **phần tử cầm canh** (sentinel). Khi đó vòng lặp chắc chắn dừng ⇒ **bỏ được** điều kiện k < N trong mỗi vòng lặp.

```
B1: k = 1
B2: M[N+1] = X          // Phần tử cầm canh
B3: IF M[k] ≠ X
      B3.1: k++
      B3.2: Lặp lại B3
B4: IF k < N+1
      Tìm thấy tại vị trí k
B5: ELSE    // k = N+1 song đó chỉ là phần tử cầm canh
      Không tìm thấy phần tử có giá trị X
B6: Kết thúc
```

```
int LinearSearch1(T M[], int N, T X)
{  int k = 0;
   M[N] = X;            // mảng phải khai báo dư 1 ô
   while (M[k] != X)
      k++;
   if (k < N)
      return (k);
   return (-1);
}
```

- **Điều kiện:** mảng phải có **ít nhất N + 1** ô.

Trường hợp	Số phép gán	Số phép so sánh	So với bản gốc
Tốt nhất	Gmin = 2	Smin = 1 + 1 = 2	3 → 2
Xấu nhất	Gmax = 2	Smax = (N + 1) + 1 = **N + 2**	2N + 2 → N + 2
Trung bình	Gavg = 2	Savg = **N/2 + 2**	N + 2,5 → N/2 + 2

- **Kết luận:** số phép so sánh giảm **gần một nửa**; nếu thời gian phép gán không đáng kể thì thuật toán cải tiến chạy nhanh hơn – nhưng vẫn **O(N)**.

2.4 Tìm nhị phân (Binary Search) – tư tưởng và thuật toán đệ quy

- **Điều kiện:** dãy **đã có thứ tự** (giả sử tăng dần).
- **Tư tưởng:** so sánh X với phần tử **giữa** M[Mid]: bằng ⇒ tìm thấy; X < M[Mid] ⇒ chỉ tìm ở **nửa đầu** (Last = Mid − 1); X > M[Mid] ⇒ chỉ tìm ở **nửa sau** (First = Mid + 1). Lặp đến khi tìm thấy hoặc **First > Last**.

```
B1: First = 1
B2: Last = N
B3: IF (First > Last)        // hết phạm vi tìm
      B3.1: Không tìm thấy
      B3.2: Thực hiện Bkt
B4: Mid = (First + Last)/2
B5: IF (X = M[Mid])
      B5.1: Tìm thấy tại vị trí Mid
      B5.2: Thực hiện Bkt
B6: IF (X < M[Mid])
      Tìm đệ quy từ First đến Last = Mid – 1
B7: IF (X > M[Mid])
      Tìm đệ quy từ First = Mid + 1 đến Last
Bkt: Kết thúc
```

```
int RecBinarySearch(T M[], int First, int Last, T X)
{  if (First > Last)
      return (-1);
   int Mid = (First + Last)/2;
   if (X == M[Mid])
      return (Mid);
   if (X < M[Mid])
      return RecBinarySearch(M, First, Mid - 1, X);
   else
      return RecBinarySearch(M, Mid + 1, Last, X);
}
int BinarySearch(T M[], int N, T X)
{  return RecBinarySearch(M, 0, N - 1, X);  }
```

- Phân tích (giáo trình): Smin = 2; **Smax = $3\log_2 N + 1$**; Savg = $\frac{1}{2}(3\log_2 N + 3)$.

2.5 Tìm nhị phân không đệ quy và phân tích

```
int NRecBinarySearch(T M[], int N, T X)
{  int First = 0;
   int Last = N - 1;
   while (First <= Last)
   {  int Mid = (First + Last)/2;
      if (X == M[Mid])
         return (Mid);
      if (X < M[Mid])
         Last = Mid - 1;
      else
         First = Mid + 1;
   }
   return (-1);
}
```

Trường hợp	Phép gán	Phép so sánh
Tốt nhất (X ở giữa)	Gmin = 3	Smin = 2
Xấu nhất (không có X)	Gmax = $2\log_2 N + 4$	Smax = $3\log_2 N + 1$
Trung bình	Gavg = $\log_2 N + 3{,}5$	Savg = $\frac{1}{2}(3\log_2 N + 3)$

- **Vì sao $\log_2 N$?** Mỗi lần lặp phạm vi tìm **giảm một nửa**: N → N/2 → N/4 → … → 1 sau ≈ $\log_2 N$ lần ⇒ **O(log N)**.
- Bản không đệ quy tiết kiệm bộ nhớ ngăn xếp và chi phí gọi hàm ⇒ thường dùng trong thực tế.

Lỗi tràn số khi tính Mid

Với mảng rất lớn, `First + Last` có thể vượt $2^{31} - 1$ ⇒ nên viết **`Mid = First + (Last - First)/2`**.

2.6 Minh họa tìm nhị phân

Ví dụ 2.3 – M = {1, 3, 4, 5, 8, 15, 17, 22, 25, 30}, tìm X = 5 (tìm thấy) – bảng của giáo trình

Lần lặp	First	Last	First > Last	Mid	M[Mid]	X = M[Mid]	X < M[Mid]	X > M[Mid]
Ban đầu	0	9	False	4	8	False	True	False
1	0	3	False	1	3	False	False	True
2	2	3	False	2	4	False	False	True
3	3	3	False	3	5	**True**		

⇒ Tìm thấy tại vị trí **3** sau 4 lần so sánh với M[Mid].

Ví dụ 2.4 – Tìm X = 7 (không tìm thấy)

Lần lặp	First	Last	Mid	M[Mid]	So sánh	Cập nhật
Ban đầu	0	9	4	8	7 < 8	Last = 3
1	0	3	1	3	7 > 3	First = 2
2	2	3	2	4	7 > 4	First = 3
3	3	3	3	5	7 > 5	First = 4
4	4	3	–	–	First > Last	Trả về **−1**

- **Phân tích:** chỉ 4 lần so sánh với phần tử giữa (≈ $\log_2 10 \approx 3{,}3$); tìm tuyến tính cần duyệt **cả 10** phần tử mới kết luận không có.

2.7 Các biến thể và lỗi thường gặp của tìm nhị phân

Ví dụ 2.5 – Tìm vị trí xuất hiện đầu tiên của X (dãy có phần tử trùng)

```
int FirstOccurrence(T M[], int N, T X)
{  int First = 0, Last = N - 1, Kq = -1;
   while (First <= Last)
   {  int Mid = First + (Last - First)/2;
      if (M[Mid] == X)
      {  Kq = Mid;          // ghi nhận
         Last = Mid - 1;    // tiếp tục tìm bên trái
      }
      else if (X < M[Mid]) Last = Mid - 1;
      else First = Mid + 1;
   }
   return Kq;
}
```

- M = {2, 5, 5, 5, 9}, X = 5: bản thường có thể trả về 2; bản này trả về **1**.

- **Tìm vị trí chèn** X để dãy vẫn tăng: khi vòng lặp kết thúc mà không tìm thấy, **First** chính là vị trí chèn (ví dụ 2.4: First = 4 ⇒ chèn 7 vào trước 8). Dùng trong **chèn nhị phân** (buổi 4).

Lỗi thường gặp	Hậu quả
Áp dụng cho dãy **chưa sắp xếp**	Kết quả sai
`while (First < Last)` thay vì `<=`	Bỏ sót trường hợp còn 1 phần tử
`First = Mid` thay vì `Mid + 1`	**Vòng lặp vô hạn** khi First = Last − 1
`Mid = (First + Last)/2` với chỉ số lớn	Tràn số nguyên
Dãy giảm dần nhưng giữ nguyên điều kiện	Tìm sai nửa

2.8 So sánh tìm tuyến tính và tìm nhị phân

N	Tuyến tính – Smax ≈ N + 2 (cầm canh)	Nhị phân – Smax ≈ $3\log_2 N + 1$	Tỉ lệ
10	12	11	≈ 1
1.000	1.002	31	≈ 32 lần
1.000.000	1.000.002	61	≈ 16.000 lần
10^9	≈ 10^9	91	≈ 10^7 lần
Tiêu chí	Tìm tuyến tính	Tìm nhị phân	
Yêu cầu dữ liệu	Không cần thứ tự	Phải **có thứ tự**	
Cấu trúc áp dụng	Mảng, DSLK, tập tin tuần tự	Mảng (truy cập ngẫu nhiên M[Mid] O(1)); không hiệu quả trên DSLK	
Độ phức tạp	O(N)	O(log N)	
Khi nào dùng	N nhỏ; dữ liệu thay đổi liên tục; chỉ tìm vài lần	N lớn; tìm **nhiều lần** trên dữ liệu ít thay đổi	

Ví dụ 2.6 – Có nên sắp xếp rồi tìm nhị phân?

Cần tìm **K** lần trên N phần tử. Tuyến tính: K·N. Sắp xếp (N log N) + K·log N. Với N = 10^6: nếu K = 1 thì tuyến tính tốt hơn ($10^6 < 2 \cdot 10^7$); nếu K = 1000 thì tuyến tính 10^9 phép, sắp xếp + nhị phân ≈ $2 \cdot 10^7$ ⇒ **nhanh gấp 50 lần**.

3.1 Đặt vấn đề tìm kiếm trên tập tin

- Dữ liệu quá lớn không nạp hết vào RAM ⇒ lưu trên **tập tin**; mỗi phần tử là một **bản ghi** có kích thước cố định (ví dụ `sizeof(SinhVien)`).
- **Chi phí chủ yếu** = số lần **đọc tập tin** (truy cập đĩa chậm hơn RAM hàng nghìn – hàng triệu lần), không phải số phép so sánh.
- **Các thao tác tập tin trong C:**

Hàm	Công dụng
`fopen(ten, "rb")`	Mở tập tin nhị phân để đọc
`fread(&x, sizeof(T), 1, f)`	Đọc 1 phần tử, trả về số phần tử đọc được
`fseek(f, k*sizeof(T), SEEK_SET)`	Nhảy đến phần tử thứ k (truy cập trực tiếp)
`ftell(f)`	Vị trí hiện tại (byte)
`fclose(f)`	Đóng tập tin

- **Kỹ thuật bộ đệm:** đọc **một khối** K phần tử vào mảng Buffer rồi tìm trên RAM ⇒ giảm số lần gọi hàm đọc từ N xuống N/K.

Ví dụ 2.7 – Hiệu quả bộ đệm

Tập tin 10^6 bản ghi. Đọc từng phần tử: **10^6** lần đọc. Bộ đệm 1.000 phần tử: **1.000** lần đọc, mỗi lần tìm tuyến tính trong mảng 1.000 phần tử trên RAM (rất nhanh).

- **Thuật toán:**

```
B1: k = 0
B2: rewind(Fd)                  // về đầu tập tin
B3: IF feof(Fd) Thực hiện B7
B4: fread(&a, sizeof(T), 1, Fd)
B5: IF (a = X) Thực hiện B8
B6: k++; Lặp lại B3
B7: k = -1                      // không tìm thấy
B8: Kết thúc  (trả về k)
```

- Kết quả là **vị trí** (thứ tự bản ghi) trong tập tin.

```
long FLinearSearch(char *FileName, T X)
{  FILE *Fd = fopen(FileName, "rb");
   if (Fd == NULL) return (-2);   // lỗi mở tập tin
   long k = 0;
   T a;
   while (fread(&a, sizeof(T), 1, Fd) == 1)
   {  if (a == X) break;
      k++;
   }
   if (feof(Fd)) k = -1;          // đọc hết mà không thấy
   fclose(Fd);
   return (k);
}
```

- **Phân tích:** tốt nhất 1 lần đọc; xấu nhất **N** lần đọc ⇒ O(N) lần truy cập đĩa.

Ví dụ 2.8 – Áp dụng: tìm sinh viên theo mã trong SV.DAT

So sánh `strcmp(a.MaSV, X) == 0` thay cho `a == X`; khi tìm thấy, dùng `fseek(Fd, k*sizeof(SinhVien), SEEK_SET)` để **sửa** bản ghi đó tại chỗ (mở với chế độ "r+b").

3.3 Tìm kiếm theo chỉ mục (Index Search)

- **Ý tưởng:** lập một **tập tin chỉ mục** gồm các cặp **(khóa, vị trí)** đã **sắp xếp theo khóa**, nhỏ hơn nhiều so với tập tin dữ liệu. Tìm trên tập tin chỉ mục (có thể tìm **nhị phân**), lấy vị trí rồi **truy cập trực tiếp** bản ghi dữ liệu bằng fseek.

Ví dụ 2.9 – Tập tin dữ liệu SV.DAT và chỉ mục SV.IDX

SV.DAT: Vị trí	MaSV	HoTen
0	SV07	Hoa
1	SV02	Nam
2	SV09	Châu
3	SV04	An

SV.IDX: MaSV	ViTri
SV02	1
SV04	3
SV07	0
SV09	2

- Tìm "SV09": tìm nhị phân trên SV.IDX ⇒ ViTri = 2 ⇒ `fseek(f, 2*sizeof(SinhVien), SEEK_SET)` ⇒ đọc **1** bản ghi.

```
typedef struct { T Key; long Pos; } IdxType;
long IndexSearch(char *IdxFile, T X)
{  FILE *Fi = fopen(IdxFile, "rb");
   IdxType a;
   while (fread(&a, sizeof(IdxType), 1, Fi) == 1)
      if (a.Key >= X) break;   // chỉ mục tăng ⇒ dừng sớm
   fclose(Fi);
   if (a.Key == X) return a.Pos;
   return -1;
}
```

Phương pháp	Số lần đọc bản ghi dữ liệu	Số lần đọc chỉ mục	Ghi chú
Tuyến tính trên SV.DAT	đến N	–	Bản ghi lớn (vd 200 byte) ⇒ đọc nhiều byte
Tuyến tính trên SV.IDX	**1**	đến N (dừng sớm vì đã sắp)	Mục chỉ mục nhỏ (vd 12 byte)
Nhị phân trên SV.IDX	**1**	$\approx \log_2 N$ (dùng fseek)	Nhanh nhất
Nạp toàn bộ SV.IDX vào RAM rồi nhị phân	**1**	1 lần đọc khối	Khi chỉ mục vừa bộ nhớ

Ví dụ 2.10 – Ước lượng với N = 10^6 sinh viên, bản ghi 200 byte, mục chỉ mục 16 byte

- Tuyến tính trên dữ liệu: trung bình đọc $5 \cdot 10^5 \times 200$ B = **100 MB**.
- Nhị phân trên chỉ mục: ≈ 20 lần đọc × 16 B + 1 bản ghi 200 B ≈ **0,5 KB**.
- **Cái giá phải trả:** tốn thêm 16 MB cho tập tin chỉ mục; mỗi lần thêm/xóa SV phải **cập nhật chỉ mục** và giữ nó có thứ tự (liên hệ: `CREATE INDEX` trong CSDL).

Liên hệ

Có thể lập **nhiều** chỉ mục cho cùng một tập tin dữ liệu (theo MaSV, theo HoTen…) mà không phải sắp xếp lại tập tin dữ liệu – ý tưởng này còn được dùng trong **sắp xếp theo chỉ mục** (buổi 5).

4.1 Tóm tắt và bài thực hành buổi 2

- **Tìm tuyến tính**: không cần thứ tự, O(N); bản **cầm canh** giảm số phép so sánh còn ≈ N/2 + 2 (trung bình).
- **Tìm nhị phân**: dãy có thứ tự, chia đôi phạm vi, O(log N); hai bản đệ quy và không đệ quy; chú ý điều kiện dừng First > Last.
- **Tìm kiếm ngoại**: tuyến tính trên tập tin (đếm số lần đọc), **theo chỉ mục** (khóa, vị trí) + fseek.

Thực hành – So sánh tìm tuyến tính và nhị phân trên mảng lớn

- 1) Sinh mảng **tăng dần** N = 10^4, 10^5, 10^6 phần tử (M[i] = 2i để biết chắc giá trị có/không có).
- 2) Cài LinearSearch, LinearSearch1 (cầm canh), RecBinarySearch, NRecBinarySearch; mỗi hàm có biến đếm `SoSanh`.
- 3) Với mỗi N, thực hiện **1.000** lần tìm với X ngẫu nhiên (một nửa có trong mảng, một nửa không); ghi **số phép so sánh trung bình** và **thời gian** (clock()).
- 4) Lập bảng, đối chiếu với công thức Savg ở trang 2.3, 2.5; nhận xét khi N tăng 10 lần thì thời gian mỗi thuật toán tăng bao nhiêu lần.

Gợi ý kiểm tra kết quả

Với N = 10^6: tuyến tính cầm canh khoảng $5 \cdot 10^5$ so sánh/lần (khi tìm thấy); nhị phân khoảng 30 – 60 so sánh/lần. Nếu số liệu lệch xa ⇒ kiểm tra lại vị trí đặt biến đếm.

4.2 Câu hỏi và bài tập

Câu hỏi

- 1) Trình bày tư tưởng tìm tuyến tính, tìm nhị phân; điều kiện áp dụng của mỗi thuật toán.
- 2) Vì sao phần tử cầm canh làm giảm số phép so sánh? Mảng cần khai báo thế nào?
- 3) Tìm nhị phân có áp dụng hiệu quả trên danh sách liên kết không? Vì sao?
- 4) Ưu, nhược điểm của tìm kiếm theo chỉ mục?

Bài tập (có gợi ý)

- 1) Mô phỏng (lập bảng First, Last, Mid) tìm nhị phân X = 25 và X = 2 trên dãy M ở ví dụ 2.1. → **Gợi ý:** X = 25: Mid = 4 (8), 7 (22), 8 (25) ⇒ vị trí **8**; X = 2: Mid = 4, 1, 0 rồi First = 1 > Last = 0 ⇒ **−1**.
- 2) Viết hàm tìm nhị phân trên dãy **giảm dần**.
- 3) Viết hàm tìm **tất cả** vị trí có giá trị X trong mảng không có thứ tự; đếm số lần xuất hiện.
- 4) Tập tin SV.DAT chứa các struct SinhVien (MaSV, HoTen, DTB): viết chương trình tạo tập tin chỉ mục SV.IDX theo MaSV (đã sắp xếp) và hàm tìm SV theo MaSV bằng tìm **nhị phân** trên SV.IDX (dùng fseek).
- 5) Tìm kiếm **nội suy** (interpolation): thay Mid = First + (X − M[First])·(Last − First)/(M[Last] − M[First]); thử trên dãy phân bố đều và so sánh số phép so sánh với nhị phân.